EXPERIMENT 1A

SETTING UP PYTHON ENVIRONMENT AND LIBRARIES – JUPYTER NOTEBOOK

Aim

To install Python, configure Jupyter Notebook, understand the notebook environment, execute Python programs, create Markdown cells, and use basic Jupyter Notebook features for Data Analytics applications.

Algorithm

Algorithm 1: Installing Python

Step 1: Download Python from

python.org Step 2: Run the installer

Step 3: Select "Add Python to

PATH" Step 4: Click Install Now

Step 5: Verify installation using:

python --version

Step 6: Observe installed Python version.

Algorithm 2: Installing Jupyter Notebook

Step 1: Open Command Prompt

Step 2: Install Jupyter

pip install notebook

Step 3: Launch Jupyter Notebook

jupyter notebook

Step 4: Browser automatically opens Notebook Dashboard.

Algorithm 3: Creating a Notebook

Step 1: Open Jupyter

Dashboard Step 2: Select New

→ Python 3 Step 3: Create

Notebook

Step 4: Rename Notebook

Step 5: Create Code and Markdown C

Result:

Python and Jupyter Notebook has been installed successfully.

Result:

Python and Jupyter Notebook has been installed successfully.

EXPERIMENT 1B

DATA ANALYSIS FRAMEWORK

Aim

To install and import libraries, import dataset and perform data pre-preprocessing using CRISP, SEMMA and KDD framework.

Algorithm:

Algorithm 1: CRISP-DM Framework

1. Business Understanding – Define the objective of analyzing the healthcare stroke dataset and identify the problem to be solved.
2. Data Understanding – Load and examine the dataset, its features, data types, missing values, and stroke distribution.
3. Data Preparation – Clean and preprocess the data by handling missing values and converting categorical variables into a suitable format.
4. Modeling – Apply an appropriate machine-learning/data-mining model to the prepared dataset.
5. Evaluation & Deployment – Evaluate the model results using suitable performance measures and present the useful findings.

Program & Output:

In [11]:
```
import pandas as pd

df = pd.read_csv(r"C:\Users\hp\Downloads\healthcare-dataset-stroke-data.csv")

print(df.head())
```

```
      id  gender   age  hypertension  heart_disease ever_married  \
0   9046    Male  67.0             0              1          Yes
1  51676  Female  61.0             0              0          Yes
2  31112    Male  80.0             0              1          Yes
3  60182  Female  49.0             0              0          Yes
4   1665  Female  79.0             1              0          Yes

      work_type Residence_type  avg_glucose_level   bmi   smoking_status  \
0       Private          Urban             228.69  36.6  formerly smoked
1  Self-employed          Rural             202.21   NaN     never smoked
2       Private          Rural             105.92  32.5     never smoked
3       Private          Urban             171.23  34.4           smokes
4  Self-employed          Rural             174.12  24.0     never smoked

   stroke
0       1
1       1
2       1
3       1
4       1
```

In [13]:
```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5110 entries, 0 to 5109
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   id                 5110 non-null   int64  
 1   gender             5110 non-null   object 
 2   age                5110 non-null   float64
 3   hypertension       5110 non-null   int64  
 4   heart_disease      5110 non-null   int64  
 5   ever_married       5110 non-null   object 
 6   work_type          5110 non-null   object 
 7   Residence_type     5110 non-null   object 
 8   avg_glucose_level  5110 non-null   float64
 9   bmi                4909 non-null   float64
 10  smoking_status     5110 non-null   object 
 11  stroke             5110 non-null   int64  
dtypes: float64(3), int64(4), object(5)
memory usage: 479.2+ KB
None
```

In [14]:
```
print(df.describe())
```

```
                 id          age  hypertension  heart_disease  \
count   5110.000000  5110.000000   5110.000000    5110.000000   
mean   36517.829354    43.226614      0.097456       0.054012   
std    21161.721625    22.612647      0.296607       0.226063   
min       67.000000     0.080000      0.000000       0.000000   
25%    17741.250000    25.000000      0.000000       0.000000   
50%    36932.000000    45.000000      0.000000       0.000000   
75%    54682.000000    61.000000      0.000000       0.000000   
max    72940.000000    82.000000      1.000000       1.000000   

       avg_glucose_level          bmi       stroke  
count        5110.000000  4909.000000  5110.000000  
mean          106.147677    28.893237     0.048728  
std            45.283560     7.854067     0.215320  
min            55.120000    10.300000     0.000000  
25%            77.245000    23.500000     0.000000  
50%            91.885000    28.100000     0.000000  
75%           114.090000    33.100000     0.000000  
max           271.740000    97.600000     1.000000  
```

```
In [6]: print(df.isnull().sum())
```

```
id                 0
gender             0
age                0
hypertension       0
heart_disease      0
ever_married       0
work_type          0
Residence_type     0
avg_glucose_level  0
bmi                0
smoking_status     0
stroke             0
dtype: int64
```

```
: print(df["stroke"].value_counts())
```

```
stroke
0    4861
1     249
Name: count, dtype: int64
```

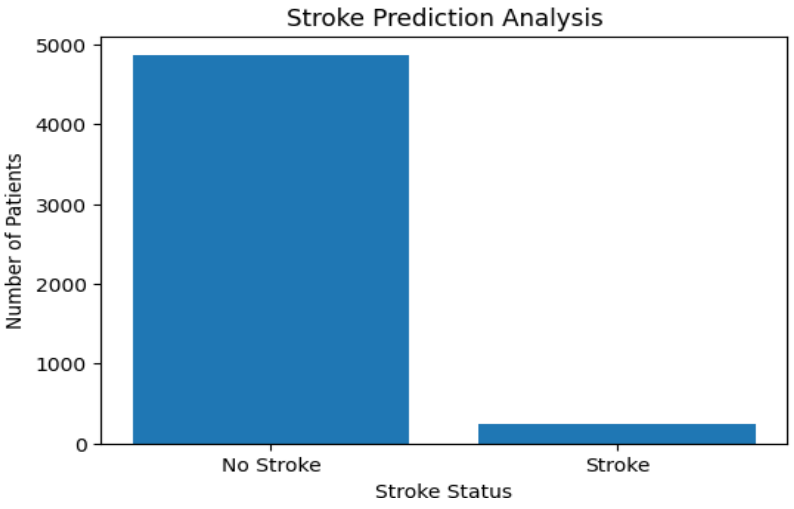
```
In [15]: import matplotlib.pyplot as plt

stroke_counts = df["stroke"].value_counts()

plt.figure(figsize=(6,4))
plt.bar(["No Stroke", "Stroke"], stroke_counts)

plt.title("Stroke Prediction Analysis")
plt.xlabel("Stroke Status")
plt.ylabel("Number of Patients")

plt.show()
```



Algorithm 2: SEMMA Framework

1. Sample – Select a representative sample from the healthcare stroke dataset.
2. Explore – Analyze the dataset by examining its features, data types, missing values, and stroke distribution.
3. Modify – Pre-process the data by handling missing values and converting categorical data into a suitable format.
4. Model – Apply a suitable data-mining/machine-learning model to the prepared dataset.
5. Assess – Evaluate the model results using suitable performance measures such as accuracy and classification report.

Program & Output:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("healthcare-dataset-stroke-data.csv")

df.head()
```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

```
print("Shape of dataset:", df.shape)

print("\nColumns:")
print(df.columns)

print("\nData types:")
print(df.dtypes)
```

```
Shape of dataset: (5110, 12)

Columns:
Index(['id', 'gender', 'age', 'hypertension', 'heart_disease', 'ever_married',
       'work_type', 'Residence_type', 'avg_glucose_level', 'bmi',
       'smoking_status', 'stroke'],
      dtype='object')

Data types:
id                     int64
gender                object
age                  float64
hypertension           int64
heart_disease          int64
ever_married          object
work_type             object
Residence_type        object
avg_glucose_level    float64
bmi                  float64
smoking_status        object
stroke                 int64
dtype: object
```

```
print("Missing values:")
print(df.isnull().sum())
```

```
Missing values:
id                   0
gender               0
age                  0
hypertension         0
heart_disease        0
ever_married         0
work_type            0
Residence_type       0
avg_glucose_level    0
bmi                201
smoking_status       0
stroke               0
dtype: int64
```

```
sample = df.sample(frac=0.8, random_state=42)

print("Original dataset:", df.shape)
print("Sample dataset:", sample.shape)

sample.head()
```

```
Original dataset: (5110, 12)
Sample dataset: (4088, 12)
```

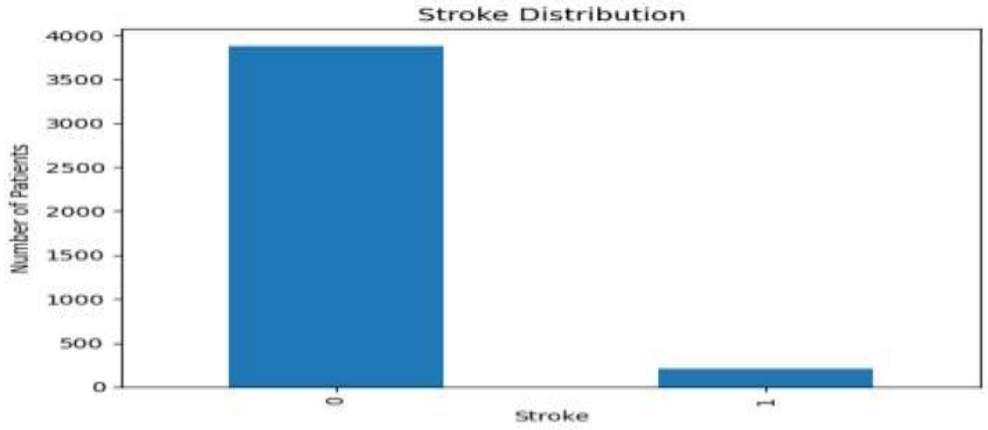
	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
4688	40041	Male	31.0	0	0	No	Self-employed	Rural	64.85	23.0	Unknown	0
4478	55244	Male	40.0	0	0	Yes	Self-employed	Rural	65.29	28.3	never smoked	0
3849	70992	Female	8.0	0	0	No	children	Urban	74.42	22.5	Unknown	0
4355	38207	Female	79.0	1	0	Yes	Self-employed	Rural	76.64	19.5	never smoked	0
3826	8541	Female	75.0	0	0	Yes	Govt_job	Rural	94.77	27.2	never smoked	0

```
print(sample["stroke"].value_counts())
```

```
stroke
0    3882
1     206
Name: count, dtype: int64
```

```
sample["stroke"].value_counts().plot(kind="bar")

plt.xlabel("Stroke")
plt.ylabel("Number of Patients")
plt.title("Stroke Distribution")
plt.show()
```




```
sample["bmi"] = sample["bmi"].fillna(sample["bmi"].median())

print("Missing values after modification:")
print(sample.isnull().sum())
```

```
Missing values after modification:
id                 0
gender             0
age                0
hypertension       0
heart_disease      0
ever_married       0
work_type          0
Residence_type     0
avg_glucose_level  0
bmi                0
smoking_status     0
stroke             0
dtype: int64
```

```
sample = sample.drop("id", axis=1)

sample.head()
```

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
4688	Male	31.0	0	0	No	Self-employed	Rural	64.85	23.0	Unknown	0
4478	Male	40.0	0	0	Yes	Self-employed	Rural	65.29	28.3	never smoked	0
3849	Female	8.0	0	0	No	children	Urban	74.42	22.5	Unknown	0
4355	Female	79.0	1	0	Yes	Self-employed	Rural	76.64	19.5	never smoked	0
3826	Female	75.0	0	0	Yes	Govt_job	Rural	94.77	27.2	never smoked	0

[10]:
```
sample = pd.get_dummies(sample, drop_first=True)

sample.head()
```

[10]:

	age	hypertension	heart_disease	avg_glucose_level	bmi	stroke	gender_Male	gender_Other	ever_married_Yes	work_type_Never_worked	work_type_Private	w
4688	31.0	0	0	64.85	23.0	0	True	False	False	False	False	
4478	40.0	0	0	65.29	28.3	0	True	False	True	False	False	
3849	8.0	0	0	74.42	22.5	0	False	False	False	False	False	
4355	79.0	1	0	76.64	19.5	0	False	False	True	False	False	
3826	75.0	0	0	94.77	27.2	0	False	False	True	False	False	

[11]:
```
X = sample.drop("stroke", axis=1)
y = sample["stroke"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (4088, 16)
y shape: (4088,)
```

[12]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

[13]:
```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)
```

[13]:

LogisticRegression

▶ Parameters

[14]:
```
from sklearn.metrics import accuracy_score, classification_report

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.9511002444987775

Classification Report:
              precision    recall  f1-score   support

           0       0.95      1.00      0.97       778
           1       0.00      0.00      0.00        40

    accuracy                           0.95       818
   macro avg       0.48      0.50      0.49       818
weighted avg       0.90      0.95      0.93       818
```

Algorithm 3: KDD Framework

1. Selection – Select the relevant attributes from the healthcare stroke dataset for analysis.
2. Preprocessing – Identify and handle missing, inconsistent, or noisy data.
3. Transformation – Transform the data into a suitable format by encoding categorical variables and preparing the features.
4. Data Mining – Apply a suitable data-mining or machine-learning algorithm to discover patterns or make predictions.
5. Interpretation/Evaluation– Analyze and evaluate the discovered results using suitable performance measures and visualizations.

Program & Output:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("healthcare-dataset-stroke-data.csv")

df.head()
```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

```
selected_data = df[
    ["gender", "age", "hypertension", "heart_disease",
     "avg_glucose_level", "bmi", "smoking_status", "stroke"]
]

selected_data.head()
```

	gender	age	hypertension	heart_disease	avg_glucose_level	bmi	smoking_status	stroke
0	Male	67.0	0	1	228.69	36.6	formerly smoked	1
1	Female	61.0	0	0	202.21	NaN	never smoked	1
2	Male	80.0	0	1	105.92	32.5	never smoked	1
3	Female	49.0	0	0	171.23	34.4	smokes	1
4	Female	79.0	1	0	174.12	24.0	never smoked	1

```
print("Missing values:")
print(selected_data.isnull().sum())
```

```
Missing values:
gender               0
age                  0
hypertension         0
heart_disease        0
avg_glucose_level    0
bmi                201
smoking_status       0
stroke               0
dtype: int64
```

	age	hypertension	heart_disease	avg_glucose_level	bmi	stroke	gender_Male	gender_Other	smoking_status_formerly smoked	smoking_status_never smoked	smoking_
0	67.0	0	1	228.69	36.6	1	True	False	True	False	
1	61.0	0	0	202.21	28.1	1	False	False	False	True	
2	80.0	0	1	105.92	32.5	1	True	False	False	True	
3	49.0	0	0	171.23	34.4	1	False	False	False	False	
4	79.0	1	0	174.12	24.0	1	False	False	False	True	

```
print("Shape after transformation:", transformed_data.shape)
print(transformed_data.dtypes)
```

```
Shape after transformation: (5110, 11)
age                              float64
hypertension                       int64
heart_disease                      int64
avg_glucose_level                float64
bmi                              float64
stroke                             int64
gender_Male                         bool
gender_Other                        bool
smoking_status_formerly smoked      bool
smoking_status_never smoked         bool
smoking_status_smokes               bool
dtype: object
```

[5]:
```
selected_data = selected_data.copy()

selected_data["bmi"] = selected_data["bmi"].fillna(
    selected_data["bmi"].median()
)

print("Missing values after preprocessing:")
print(selected_data.isnull().sum())
```

```
Missing values after preprocessing:
gender               0
age                  0
hypertension         0
heart_disease        0
avg_glucose_level    0
bmi                  0
smoking_status       0
stroke               0
dtype: int64
```

[6]:
```
transformed_data = pd.get_dummies(
    selected_data,
    drop_first=True
)

transformed_data.head()
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X = transformed_data.drop("stroke", axis=1)
y = transformed_data["stroke"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
from sklearn.metrics import accuracy_score, classification_report

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.9393346379647749

Classification Report:
              precision    recall  f1-score   support

           0       0.94      1.00      0.97       960
           1       0.00      0.00      0.00        62

    accuracy                           0.94      1022
   macro avg       0.47      0.50      0.48      1022
weighted avg       0.88      0.94      0.91      1022
```

Result:

The healthcare stroke dataset was successfully loaded, pre-processed, analyzed, and visualized using the CRISP-DM, SEMMA, and KDD data mining frameworks. The dataset was successfully prepared for data mining and the results were evaluated using a suitable machine-learning model.